

CyberAudit

Frontend Tests (Vitest)

40 tests passing / validators (21) / StatusBadge (8) / useAuth (5) / ProtectedRoute (6)

Project	CyberAudit Frontend (React + Vite)
Test framework	Vitest + @testing-library/react + jsdom
Test directory	src/__tests__/
Test files	validators.test.js / StatusBadge.test.jsx / useAuth.test.jsx / ProtectedRoute.test.jsx
Total tests	40 passing, 0 failing
Coverage	11.6% statement coverage (targeted unit tests)
Subjects covered	validators (pure functions) / UI component render + classes / hook context / route guard RBAC
Test helpers	ConsumerComponent, renderWithAuth, mockCtx, 4 auth state constants
Date	June 2026 / Juin 2026

1. Overview : 40 Tests across 4 Files

1. Vue d'ensemble : 40 tests en 4 fichiers

The frontend test suite targets the four most critical units: pure validation logic, the status display component, the auth hook, and the route guard. Each test file is focused and self-contained, no shared global fixtures. The 11.6% statement coverage reflects intentional scope rather than incomplete coverage.

La suite de tests frontend cible les quatre unités les plus critiques : la logique de validation pure, le composant d'affichage de statut, le hook d'auth et le garde de route. Chaque fichier de test est focalisé et autonome, pas de fixtures globales partagés. Le taux de couverture de 11,6% reflète un périmètre intentionnel.

File	Subject	Tests	What is verified EN / FR
validators.test.js	utils/validators.js	21	Pure functions: no DOM, no React. Fast unit tests. Fonctions pures : pas de DOM. Tests unitaires rapides.
StatusBadge.test.jsx	components/dashboard/StatusBadge.jsx	8	Render text + CSS classes per status. Fail-safe default. Rendu texte + classes CSS par statut. Défaut sécurise.
useAuth.test.jsx	hooks/useAuth.js	5	Context consumption, hasRole(), error outside Provider. Consommation contexte, hasRole(), erreur hors Provider.
ProtectedRoute.test.jsx	components/auth/ProtectedRoute.jsx	6	RBAC gate -- loading / redirect / allow / deny by role. Portail RBAC -- chargement / redirect / acces / refus.
TOTAL	4 files	40	0 en echec

2. Test Infrastructure (Vitest + Testing Library)

2. Infrastructure de test (Vitest + Testing Library)

Vitest runs in jsdom environment, simulating the browser DOM without a real browser. `@testing-library/react` provides `render()`, `screen`, and `renderHook()` which test components the way users interact with them, by text content and accessible attributes, not implementation details.

Vitest s'exécute dans l'environnement jsdom, simulant le DOM du navigateur sans vrai navigateur.

`@testing-library/react` fournit `render()`, `screen` et `renderHook()` qui testent les composants comme les utilisateurs les utilisent, par contenu textuel et attributs accessibles, pas les détails d'implémentation.

```
// vitest.config.js
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom', // browser DOM simulation
    globals: true, // describe/it/expect without import
    setupFiles: ['./src/setupTests.js'], // @testing-library/jest-dom matchers
    coverage: {
      provider: 'v8',
      reporter: ['text', 'lcov'],
    },
  },
})
```

```
// src/setupTests.js
import '@testing-library/jest-dom' // adds .toBeInTheDocument(), .toContain(),
etc.
```

jsdom environment	Simulates window, document, DOM APIs. Allows <code>render()</code> without a browser process. Simule window, document, les APIs DOM. Permet <code>render()</code> sans processus navigateur.
<code>@testing-library/react</code>	<code>render()</code> mounts component into jsdom. <code>screen</code> queries DOM by text/role/testid. <code>render()</code> monte le composant dans jsdom. <code>screen</code> interroge le DOM par texte/role/testid.
<code>renderHook()</code>	Mounts a minimal wrapper just to call the hook. Returns <code>result.current</code> for assertions. Monte un wrapper minimal pour appeler le hook. Retourne <code>result.current</code> .
jest-dom matchers	<code>toBeInTheDocument()</code> , <code>toContain(class)</code> , etc. Semantic assertions on DOM nodes.

	Assertions sémantiques sur les nœuds DOM.
Run commands	<code>vitest --run (CI, no watch) / vitest --coverage (coverage report)</code> <code>vitest --run (CI) / vitest --coverage (rapport de couverture)</code>

3. validators.test.js : 21 Tests

3. validators.test.js : 21 tests

validators.js is the most thoroughly tested file with 21 tests across 5 describe blocks. These are pure function tests, no DOM, no React, no imports from the browser API. They run in microseconds and cover the full input space: valid values, empty values, null, whitespace, and boundary cases.

validators.js est le fichier le plus testé avec 21 tests en 5 blocs décrits. Ce sont des tests de fonctions pures, pas de DOM, pas de React. Ils s'exécutent en microsecondes et couvrent l'espace d'entrée complet : valeurs valides, vides, null, espaces et cas limites.

isRequired : 4 tests

```
describe('isRequired', () => {
  it('retourne true pour une chaîne non vide', () =>
    expect(isRequired('hello')).toBe(true))
  it('retourne false pour une chaîne vide', () =>
    expect(isRequired('')).toBe(false))
  it('retourne false pour une chaîne d\'espaces', () => expect(isRequired('
')).toBe(false))
  it('retourne false pour null', () =>
    expect(isRequired(null)).toBe(false))
})

// isRequired contract: value is non-null AND non-whitespace-only
```

isEmailValid : 4 tests

```
describe('isEmailValid', () => {
  it('accepte un email valide', () =>
    expect(isEmailValid('marie@example.com')).toBe(true))
  it('refuse un email sans @', () =>
    expect(isEmailValid('marieexample.com')).toBe(false))
  it('refuse un email sans domaine', () =>
    expect(isEmailValid('marie@')).toBe(false))
  it('refuse une chaîne vide', () => expect(isEmailValid('')).toBe(false))
})

// Regex: requires @, non-empty local, non-empty domain
```

isPasswordStrong : 5 tests

```
describe('isPasswordStrong', () => {
  it('accepte fort (10+ chars, MAJ, min, chiffre, spécial)',
    () => expect(isPasswordStrong('Security!X')).toBe(true))
  it('refuse trop court (< 10)',
    () => expect(isPasswordStrong('Sec1!')).toBe(false))
  it('refuse sans majuscule',
    () => expect(isPasswordStrong('security!x')).toBe(false))
  it('refuse sans chiffre',
    () => expect(isPasswordStrong('Security!X')).toBe(false))
  it('refuse sans caractère spécial',
    () => expect(isPasswordStrong('SecuritylX')).toBe(false))
})

// Rules: length >= 10, /[A-Z]/, /[0-9]/, /^[A-Za-z0-9]/
```

validateLoginForm : 4 tests | validateRegisterForm : 4 tests

```
describe('validateLoginForm', () => {
  it('retourne {} pour des identifiants valides', () => {
    expect(validateLoginForm({email:'marie@example.com',password:'anypassword'})).toEqual({})
  })
  it('signale email manquant', () =>
    expect(validateLoginForm({email:'',password:'p'}).email).toBe(MISSING_FIELD_MESSAGE))
  it('signale email mal formaté', () =>
    expect(validateLoginForm({email:'notanemail',password:'p'}).email).toBeTruthy())
  it('signale mot de passe manquant', () =>
    expect(validateLoginForm({email:'x@y.com',password:''}).password).toBe(MISSING_FIELD_MESSAGE))
})

describe('validateRegisterForm', () => {
  const valid = {companyName:'Compta Plus',firstName:'Marie',lastName:'Dupont',
    email:'marie@example.com',password:'Security!X'}
  it('retourne {} pour formulaire complet et valide', () => {
    expect(validateRegisterForm(valid)).toEqual({})
  })
  it('signale les 3 champs vides (company, first, last)', () => {
    const errs =
    validateRegisterForm({...valid,companyName:'',firstName:'',lastName:''})
    expect(errs.companyName).toBe(MISSING_FIELD_MESSAGE)
    expect(errs.firstName).toBe(MISSING_FIELD_MESSAGE)
    expect(errs.lastName).toBe(MISSING_FIELD_MESSAGE)
  })
  it('signale email invalide', () =>
    expect(validateRegisterForm({...valid,email:'bad-email'}).email).toBeTruthy())
  it('signale mot de passe faible', () =>
    expect(validateRegisterForm({...valid,password:'weak'}).password).toBeTruthy())
})

// {} means no errors -- used to enable/disable Submit button in the form
```

Test group	#	Return type	Design note EN / FR
isRequired	4	boolean	Handles null + whitespace, common form edge cases. Gere null + espaces, cas limites courants des formulaires.
isEmailValid	4	boolean	Regex boundary: sans @, sans domaine, vide. Limites regex : sans @, sans domaine, vide.
isPasswordStrong	5	boolean	Each rule tested independently, ensures no single rule masks another. Chaque règles testées indépendamment.
validateLoginForm	4	{ } or {field:msg}	{ } = valid. errors.field = message string. Matches form's disabled-button logic. { } = valide. Correspond a la logique bouton désactive.
validateRegisterForm	4	{ } or {field:msg}	Spread ...valid to change one field at a time. Clean mutation testing pattern. Spread ...valid pour changer un champ a la fois.

4. StatusBadge.test.jsx : 8 Tests

4. StatusBadge.test.jsx : 8 tests

StatusBadge maps an audit request status string to a localised label and Tailwind colour classes. The 8 tests verify all 4 status labels, 3 colour class patterns, and the fail-safe fallback (unknown status defaults to Pending instead of crashing).

StatusBadge traduit un statut de demande d'audit en libelle localise et classes couleur Tailwind. Les 8 tests verifient les 4 libelles de statut, 3 patterns de classe couleur, et le fallback securise (statut inconnu retombe sur Pending au lieu de planter).

```
import { render, screen } from '@testing-library/react'
import StatusBadge from '../components/dashboard/StatusBadge.jsx'

describe('StatusBadge', () => {
  // — Text labels (4 tests) —————
  it('affiche "Pending" pour pending', () => {
    render(<StatusBadge status='pending' />)
    expect(screen.getByText('Pending')).toBeInTheDocument()
  })
  it('affiche "In Progress" pour in_progress', () => {
    render(<StatusBadge status='in_progress' />)
    expect(screen.getByText('In Progress')).toBeInTheDocument()
  })
  it('affiche "Completed" pour completed', () => {
    render(<StatusBadge status='completed' />)
    expect(screen.getByText('Completed')).toBeInTheDocument()
  })
  it('affiche "Archived" pour archived', () => {
    render(<StatusBadge status='archived' />)
    expect(screen.getByText('Archived')).toBeInTheDocument()
  })

  // — Fail-safe (1 test) —————
  it('retombe sur Pending pour statut inconnu', () => {
    render(<StatusBadge status='unknown_status' />)
    expect(screen.getByText('Pending')).toBeInTheDocument()
  })

  // — CSS colour classes (3 tests) —————
  it('classes bleues pour in_progress', () => {
    render(<StatusBadge status='in_progress' />)
    expect(screen.getByText('In Progress').className).toContain('blue')
  })
  it('classes vertes pour completed', () => {
    render(<StatusBadge status='completed' />)
    expect(screen.getByText('Completed').className).toContain('green')
  })
  it('classes grises pour archived', () => {
    render(<StatusBadge status='archived' />)
    expect(screen.getByText('Archived').className).toContain('gray')
  })
})

// Pattern: render -> getByText -> check .toBeInTheDocument() or .className
```


Status	Label displayed	Tailwind colour	Test assertion EN / FR
pending	Pending	yellow (default)	toBelInTheDocument(), text present. Texte present.
in_progress	In Progress	blue	toBelInTheDocument() + className.toContain('blue'). Texte present + classe bleue.
completed	Completed	green	toBelInTheDocument() + className.toContain('green'). Texte present + classe verte.
archived	Archived	gray	toBelInTheDocument() + className.toContain('gray'). Texte present + classe grise.
unknown_status	Pending (fail-safe)	yellow (default)	Falls back to Pending, component never crashes on unknown input. Retour à Pending, le composant ne plante pas.

getByText() query	Finds the DOM node by its visible text. Tests what the user sees, not the component's internal structure. Trouve le noeud DOM par son texte visible. Teste ce que l'utilisateur voit.
.toBelInTheDocument()	jest-dom matcher. Asserts the element exists and is attached to the jsdom document. Matcher jest-dom. Vérifie que l'élément existe dans le document jsdom.
.className.toContain()	String match on the rendered class attribute. Tolerant of multi-class strings like 'bg-blue-100 text-blue-800'. Correspondance de chaine sur l'attribut class rendu. Tolérant des classes multiples.
Fail-safe test	Tests defensive programming, the component handles unknown input gracefully without throwing. Teste la programmation défensive gestion de l'entree inconnue sans exception.

5. useAuth.test.jsx : 5 Tests

5. useAuth.test.jsx : 5 tests

useAuth() is a React hook that reads AuthContext. Testing it requires wrapping with AuthContext.Provider. Two approaches are used: render() with a ConsumerComponent for DOM assertions, and renderHook() for direct hook state inspection. The fifth test confirms the hook throws a descriptive error when used outside a Provider.

useAuth() est un hook React qui lit AuthContext. Le tester requiert un wrapper avec AuthContext.Provider. Deux approches sont utilisées : render() avec un ConsumerComponent pour les assertions DOM, et renderHook() pour l'inspection directe de l'état du hook. Le cinquième test confirme que le hook lance une erreur descriptive hors Provider.

```
// — Test helper: minimal consumer component —————
function ConsumerComponent() {
  const { user, isAuthenticated } = useAuth()
  return (
    <div>
      <span data-testid='auth'>{String(isAuthenticated)}</span>
      <span data-testid='role'>{user?.role ?? 'none'}</span>
    </div>
  )
}

// — Mock context value —————
const mockCtx = {
  user: { role: 'client', email: 'marie@example.com' },
  token: 'fake.token', ready: true, isAuthenticated: true,
  login: () => {}, register: () => {}, logout: () => {},
  hasRole: (role) => role === 'client', // only 'client' returns true
}

// — Tests —————
describe('useAuth', () => {
  it('expose isAuthenticated=true dans un Provider connecté', () => {
    render(<AuthContext.Provider value={mockCtx}><ConsumerComponent
  /></AuthContext.Provider>)
    expect(screen.getByTestId('auth').textContent).toBe('true')
  })

  it('expose le rôle de l\'utilisateur', () => {
    render(<AuthContext.Provider value={mockCtx}><ConsumerComponent
  /></AuthContext.Provider>)
    expect(screen.getByTestId('role').textContent).toBe('client')
  })

  it('hasRole retourne true pour le bon rôle', () => {
    const { result } = renderHook(() => useAuth(), {
      wrapper: ({ children }) => (
        <AuthContext.Provider value={mockCtx}>{children}</AuthContext.Provider>
      ),
    })
    expect(result.current.hasRole('client')).toBe(true)
  })
})
```

```
it('hasRole retourne false pour un rôle incorrect', () => {
  const { result } = renderHook(() => useAuth(), { wrapper: ... })
  expect(result.current.hasRole('admin')).toBe(false)
})

it('lève une erreur si utilisé hors d\'un AuthProvider', () => {
  expect(() => renderHook(() => useAuth())).toThrow('useAuth must be used')
})

// Two-role hasRole() check: 'client' -> true, 'admin' -> false in the same
mockCtx
```

ConsumerComponent helper	Minimal React component that reads the hook and exposes values in data-testid spans. Tests DOM output, not hook internals. Composant minimal qui lit le hook et expose les valeurs dans des spans. Teste la sortie DOM.
mockCtx hasRole	hasRole: (role) => role === 'client'. Both true and false paths tested with the same context object. Les deux chemins true et false testés avec le même objet de contexte.
renderHook + wrapper	The wrapper prop provides the Provider context for the hook. result.current gives direct access to hook return value. Le wrapper fournit le contexte Provider. result.current donne l'accès direct.
Error outside Provider	useAuth() throws when AuthContext is undefined (no Provider). The test asserts the message string to confirm it's intentional. useAuth() lance quand AuthContext est undefined. Le test vérifie le message pour confirmer le caractère intentionnel.

6. ProtectedRoute.test.jsx : 6 Tests

6. ProtectedRoute.test.jsx : 6 tests

ProtectedRoute is the RBAC gate for all authenticated pages. It reads from AuthContext (ready flag, isAuthenticated, user.role) and either renders the child route, shows a loader, or redirects. Testing requires MemoryRouter to simulate navigation without a real browser history.

ProtectedRoute est le portail RBAC de toutes les pages authentifiées. Il lit AuthContext (flag ready, isAuthenticated, user.role) et soit affiche le contenu enfant, affiche un loader, soit redirige. Le test requiert MemoryRouter pour simuler la navigation sans historique de navigateur réel.

```
// — 4 named auth states used across all tests —————
const NOT_READY = { ready: false, isAuthenticated: false, user: null }
const ANON       = { ready: true,  isAuthenticated: false, user: null }
const CLIENT     = { ready: true,  isAuthenticated: true,  user: { role: 'client' } }
const ADMIN      = { ready: true,  isAuthenticated: true,  user: { role: 'admin'  } }

// — renderWithAuth helper —————
function renderWithAuth(contextValue, allowedRoles = []) {
  return render(
    <AuthContext.Provider value={contextValue}>
      <MemoryRouter initialEntries={['/protected']}>
        <Routes>
          <Route path='/login'      element={<div>Page Login</div>} />
          <Route path='/'          element={<div>Page Accueil</div>} />
          <Route element={<ProtectedRoute allowedRoles={allowedRoles} />>
            <Route path='/protected' element={<div>Contenu Protégé</div>} />
          </Route>
        </Routes>
      </MemoryRouter>
    </AuthContext.Provider>
  )
}
```

```
describe('ProtectedRoute', () => {
  it('affiche un loader quand ready=false', () => {
    renderWithAuth(NOT_READY)
    expect(screen.getByText('Loading...')).toBeInTheDocument()
  })

  it('redirige vers /login si non authentifié', () => {
    renderWithAuth(ANON)
    expect(screen.getByText('Page Login')).toBeInTheDocument()
  })

  it('affiche le contenu si authentifié sans restriction de rôle', () => {
    renderWithAuth(CLIENT)
    expect(screen.getByText('Contenu Protégé')).toBeInTheDocument()
  })

  it('affiche le contenu si le rôle correspond à allowedRoles', () => {
    renderWithAuth(ADMIN, ['admin'])
  })
})
```

```

    expect(screen.getByText('Contenu Protégé')).toBeInTheDocument()
  })

  it('redirige vers / si le rôle ne correspond pas à allowedRoles', () => {
    renderWithAuth(CLIENT, ['admin']) // client tries admin route
    expect(screen.getByText('Page Accueil')).toBeInTheDocument()
  })

  it('affiche le contenu si allowedRoles contient plusieurs rôles valides', () => {
    {
      renderWithAuth(CLIENT, ['client', 'admin'])
      expect(screen.getByText('Contenu Protégé')).toBeInTheDocument()
    }
  })
})

// MemoryRouter + initialEntries simulates navigating to /protected at mount time

```

#	Auth state	allowedRoles	Expected	Security meaning EN / FR
1	NOT_READY	[]	Loading...	Auth still initialising show spinner, never redirect prematurely. Auth en cours spinner, jamais de redirect premature.
2	ANON	[]	-> /login	Unauthenticated user redirected to login. Utilisateur non-auth redirige vers login.
3	CLIENT	[] (any role)	Protected content	Authenticated, no role restriction, content shown. Authentifié, pas de restriction, contenu affiche.
4	ADMIN	['admin']	Protected content	Role matches allowedRoles, access granted. Role dans allowedRoles, acces autorise.
5	CLIENT	['admin']	-> /	Client tries admin route, redirected to home (not 403). Client essaie route admin, redirige vers accueil.
6	CLIENT	['client','admin']	Protected content	Multi-role list, client is in the list, access granted. Liste multi-roles, client dans la liste, accès autorise.

7. Coverage Report : 11.6% Statement

7. Rapport de couverture : 11,6% statement

11.6% statement coverage is deliberately low; the tests target the four units with the highest risk-to-complexity ratio. The validators and route guard are tested exhaustively; the remaining 88.4% covers pages, API calls, and context logic that would require full integration tests or E2E tests to meaningfully cover.

11,6% de couverture statement est délibérément faible, les tests ciblent les quatre unités avec le ratio risque/complexité le plus élevé. Les validateurs et le garde de route sont testés exhaustivement ; les 88,4% restants couvrent les pages, les appels API et la logique de contexte qui nécessiteraient des tests d'intégration ou E2E pour être significativement couverts.

Module	Coverage	Tests	Note EN / FR
utils/validators.js	~100%	21	All branches tested. Toutes les branches testées.
components/dashboard/StatusBadge.jsx	~100%	8	All statuses + fail-safe. Tous les statuts + fail-safe.
hooks/useAuth.js	~100%	5	All hook paths tested. Tous les chemins du hook.
components/auth/ProtectedRoute.jsx	~100%	6	All 4 auth states + role matching. 4 états auth + matching de rôle.
Pages (Login, Dashboard, etc.)	0%	0	Require E2E or integration tests. Nécessitent E2E ou tests d'intégration.
API layer (axiosClient, services)	0%	0	Covered by backend pytest suite. Couverts par la suite pytest backend.

AuthContext provider logic	0%	0	Mocked via mockCtx in hook tests. Mocke via mockCtx dans les tests hook.
OVERALL	11.6%	40	Targeted unit coverage. Couverture ciblée des unités.

Run commands

```
# Run all frontend tests (single pass, CI mode)
cd ~/portfolio/cyberaudit/frontend
npx vitest --run

# Run with coverage report
npx vitest --run --coverage

# Run one specific file
npx vitest --run src/__tests__/validators.test.js

# Watch mode (development)
npx vitest

# Expected output:
# Test Files  4 passed (4)
# Tests      40 passed (40)
# Duration   ~1s
```

8. Design & Security Summary

8. Recapitulatif conception et securite

Pattern	Detail EN / FR
Pure function tests first	21/40 tests target pure functions (validators). No DOM setup, no mocking fastest and most reliable tests. 21/40 tests ciblent des fonctions pures. Pas de DOM, pas de mock, tests les plus rapides et fiables.
Password strength rules	Each rule tested independently: length / uppercase / digit / special char. No single rule masks another. Chaque regle testee independamment pour éviter qu'une règle masque une autre.
{ } = no errors pattern	validateLoginForm + validateRegisterForm return {} on success. Matches the form's Object.keys(errors).length === 0 guard. Retournent {} en succes. Correspond au garde Object.keys(errors).length === 0 du formulaire.
StatusBadge fail-safe	Unknown status falls back to Pending instead of crashing. Defensive UI - - backend can add statuses without breaking the frontend. Statut inconnu retombe sur Pending. UI defensive -- le backend peut ajouter des statuts sans casser le frontend.
useAuth outside Provider	Hook throws 'useAuth must be used...', descriptive error catches missing Provider wrapping during development. Le hook lance une erreur descriptive, détecte l'oubli de Provider pendant le développement.
ProtectedRoute ready flag	NOT_READY state shows Loading... -- prevents redirect before JWT token is parsed from localStorage. Etat NOT_READY affiche Loading... évite la redirige avant le parsing du JWT.
Role mismatch -> / not 403	Client accessing admin route is redirected to / (home), not a 403 page. Silent redirect is a UX choice. Client sur route admin redirige vers / (accueil), pas une page 403. Redirect silencieuse.
Multi-role allowedRoles	allowedRoles=['client','admin'] allows both roles. Single-element and multi-element arrays both tested. AllowedRoles multi-valeurs permet plusieurs roles. Tableaux a 1 et N elements testes.
MemoryRouter for routing	MemoryRouter with initialEntries=['/protected'] simulates navigation without browser history API. Simule la navigation sans l'API history du navigateur.

mockCtx pattern	Single mock object tests both true and false paths for hasRole(). Keeps test count low without losing coverage. Un seul objet mock teste les deux chemins de hasRole(). Peu de tests, couverture maintenue.
jsdom + testing-library	No real browser needed, tests run in Node. Queries by visible text, not CSS selectors or component names. Pas de vrai navigateur. Requetes par texte visible, pas par sélecteur CSS.